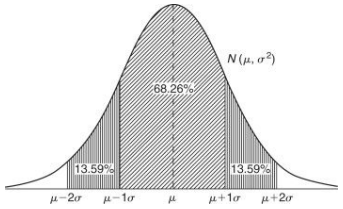
Lab 13: Normalizing Flows (Part 1) and Stable Diffusion (Part 2)



Part 1 (Normalizing Flow):

Sequence of steps that tranforms the data into a normal(Gaussian) distribution, while keeping exact track of how **probability density changes** along the way (via the **Jacobian determinant**)

Objectives for Part1 :

- Implement RealNVP - a specific type of Normalizing flow - and train it in the MNIST handwritten digit dataset
- Final Model: take random Gaussian Noise and turn it into images that look like real digits

Approach:

- Start with VAEs and see where they fall short and why we need Normalizing Flows

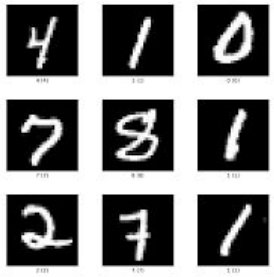
Attributions:

- This lab is adapted from the "Normalizing Flows for Image Modeling" tutorial by Phillip Lippe (UvA Deep Learning Course).
- Original Link: https://uvadlc-notebooks.readthedocs.io/en/latest/

Context:

- We have 60,000 MNIST images and we want the model to:

1. Understand what the real digits look like (learn the distribution)
2. Generate new image that looks like it came from this same distribution



A plain CNN can't do this. It maps images to labels, not the other way around.
So, We need a model that understands this space of valid images

VAEs **(2 separate set of weights)**:

- Encoder: IMAGE → Z(compressed latent vector) → μ: where in the latent space the image lives; σ: how uncertain are we about this location

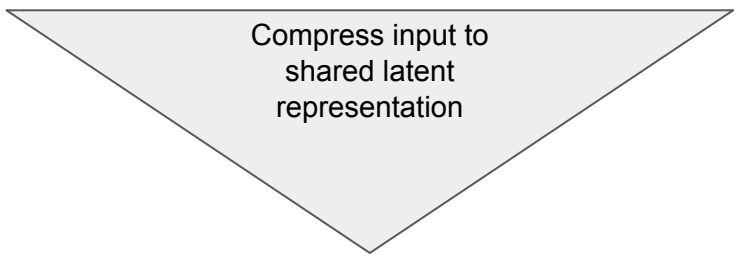
- Decoder: Z → Reconstructed image

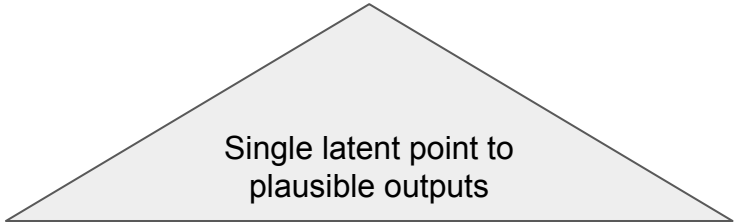
VAE Loss: minimize (Reconstruction Loss + KL Divergence)

- **Reconstruction Loss**: make output look like the input (forces the encoder to preserve enough information)
- **KL Divergence** *(a.k.a regularizer on the latent space)*:
    1. measures how far the encoder's Gaussian blobs are from N(0,1).
    2. Minimizing it => pushing these blobs to N(0,1)
    3. Sample random noise from N(0,1) at the generation time and be confident that it lands somewhere meaningful (successful generation)

Limitations:

- To compute true probability p(x) of an image, we need to consider every possible z that could generate it (entire latent space)
- Integrating over it => mathematically intractable
- So, lower bound log p(x) optimization
- Never maximizing the actual likelihood of the image
- Results: Blurry Reconstructions, No exact density, and 2 separate independent networks





Limitations of VAEs:

1. Approximations (instead of exact p(x)) -> `log p(x)` **>=** `E[log p(x|z)] - KL(q(z|x) || p(z))`
2. Need for 2 separate components

HOW NORMALIZING FLOWS SOLVE THIS?

Instead of a separate encoder and decoder, RealNVP learns one invertible transformation - a function that maps images to Gaussian noise, with a mathematically guaranteed inverse that maps noise back to images.

f: Z -> X such that x = f(z) and z = f^{-1}(x)

To compute probability density p(x), RealNVP uses **"Change of Variables formula"**

$$p(x) = p(f^{-1}(x)) \cdot \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right|$$

In our MNIST example: 28x28 image grids (784 pixels),

- x **(784, 1)** -> [0.0, 0.0, 0.04, 0.78, 0.99, 0.85, 0.12, 0.0, ..., 0.0]. Non-zeros -> penstrokes
- f^{-1}(x) **(784,1)** -> [0.31, -1.2, 0.87, -0.04, 1.1, -0.67, ..., 0.22]. random Gaussian noise: no digit structure left
- p(f^{-1}(x)) **(1)** -> how likely are these 784 numbers under a standard normal distribution N(0,I)
- ∂f^{-1}(x)/∂x - the Jacobian **(784,784)** -> (i,j) : if I nudge pixel j of MNIST image x by tiny amount, how much does output i of z change?
- det(Jacobian) **(1)** -> did the transformation expand or compress the space around this image x
- p(x) -> How likely is this specific arrangement of 784 pixel values to be a real handwritten digit?

Log-based formula:

$$\log p(x) = \log p(z) + \log \left| \det \frac{\partial z}{\partial x} \right|$$

where $\frac{\partial z}{\partial x}$ is the Jacobian Matrix of the inverse transformation.

Composing Flows

To model complex data like images, a single transformation is not enough. We compose a sequence of K invertible transformations:

$$z_K = f_K \circ \cdots \circ f_1(z_0), \quad \text{where } z_0 = x \text{ and } z_K = z$$

The log-likelihood for the full chain is the sum of the log-determinants of each step:

$$\log p(x) = \log p(z_K) + \sum_{k=1}^{K} \log \left| \det \frac{\partial f_k}{\partial z_{k-1}} \right|$$

Gist: One transformation is not powerful enough to map a complex distribution of handwritten digits all the way to a clean Gaussian. So we stack K transformations. Each one does a little bit of the work.

IN THE MNIST EXAMPLE:

- Z_0 **(784,1)** => [0.0, 0.0, 0.04, 0.78, 0.99, 0.85, 0.12, 0.0, ..., 0.0]: raw MNIST image x
- Z_1**(784,1)** => [0.12, -0.34, 0.67, 0.91, -0.45, 0.23, ..., -0.11]: image after the first transformation f_1. Still 784 numbers. Still has some structure from the digit but slightly more Gaussian-looking than z_0.
- Z_k**(784,1)** => [0.31, -1.2, 0.87, -0.04, 1.1, -0.67, ..., 0.22]: final output after all K transformations. This should now look exactly like it was sampled from N(0,I). No digit structure remaining. Just 784 numbers that look like pure Gaussian noise.
- **Det in composite flow:** How much did transformation k stretch or compress space? One scalar per layer. Because we are in log space, these just add up across layers instead of multiplying

KEY OBSERVATIONS:

- Jacobian is (784,784) => Computing its determinant normally costs O(784^3). That is too expensive to do at every training step for every image.

SOLUTION: Affine Coupling Layer

- Split Image x into (x_a, x_b). Imagine the 28×28 image like a chessboard. x_a is all the "black squares" - every pixel at position where (row + col) is even. 392 pixels total. X_b are the “white squares”

1. **Forward (Inference $x \to z$):**

$$z_a = x_a$$
$$z_b = x_b \odot \exp(s(x_a)) + t(x_a)$$

Here, $s(\cdot)$ (scale) and $t(\cdot)$ (translation) are arbitrary neural networks (e.g., ResNets). They do **not** need to be invertible themselves.

2. **Inverse (Generation $z \to x$):**

$$x_a = z_a$$
$$x_b = (z_b - t(z_a)) \odot \exp(-s(z_a))$$

3. **Jacobian Determinant:** Because z_a does not depend on x_b, the Jacobian matrix is triangular. The determinant is simply the product of the diagonal elements:

$$\log \left| \det \frac{\partial z}{\partial x} \right| = \sum \log |\exp(s(x_a))| = \sum s(x_a)$$

This is extremely efficient to compute.

Key Observations (Forward):

1. Half the pixels are frozen. Not transformed at all in this layer. This is the key design decision that makes the Jacobian triangular.(Eq 1)
2. s: It is just predicting how much to scale each pixel in x_b.
3. We exponentiate the scale values elementwise. This ensures every scale factor is strictly positive - you cannot scale a pixel by a negative number and maintain invertibility.
4. This is the actual transformation happening to the "white square" pixels of your MNIST image
5. To undo the forward pass we just subtract the translation then divide by the scale. We get back the exact original pixel values. Perfect reconstruction, no approximation.
6. z_a does not depend on x_b at all. So the bottom-left quadrant of the full [784×784] Jacobian is all zeros. A matrix with a zero block like that is triangular. The determinant of a triangular matrix is just the product of its diagonal entries (784)